

Chapter 4 Notes: Classification

Object-Oriented Analysis and Design Study Notes

CSE 460: Software Analysis and Design

Contents

1	Chapter Purpose	2
2	Why Classification Matters in Object-Oriented Design	2
2.1	Classification as Ordering Knowledge	2
2.2	Problem the Concept Solves	2
2.3	Classification Affects the Entire Design	2
3	Discovery, Invention, and Sameness	3
3.1	Discovery	3
3.2	Invention	3
3.3	Classification as Finding Sameness	3
4	Why Classification Is Difficult	4
4.1	Boundaries Are Often Fuzzy	4
4.2	There Is No Perfect Classification	4
4.3	Classification Depends on the Observer	4
4.4	Classification Requires Creative Insight	4
5	The Incremental and Iterative Nature of Classification	5
5.1	Classification Evolves During Design	5
5.2	Common Refactorings of Classification	5
5.3	Why Iteration Is Necessary	5
6	Three Approaches to Classification	6
6.1	Overview	6
6.2	Classical Categorization	6
6.2.1	Definition	6
6.2.2	Software Interpretation	6
6.2.3	Example	6
6.2.4	Benefits	6
6.2.5	Limitations	7
6.3	Conceptual Clustering	7
6.3.1	Definition	7
6.3.2	Software Interpretation	7
6.3.3	Example: “Reportable Event”	7
6.3.4	Benefits	7
6.3.5	Tradeoff	7

6.4	Prototype Theory	7
6.4.1	Definition	7
6.4.2	Software Interpretation	8
6.4.3	Interactional Properties	8
6.4.4	Software Engineering Use	8
7	Applying Classification Theory to Object-Oriented Analysis	8
7.1	Recommended Order of Use	8
7.2	Conceptual Workflow	9
7.3	Engineering Rule of Thumb	9
8	Object-Oriented Analysis	9
8.1	Analysis Versus Design	9
8.2	Why Naive Analysis Fails	9
9	Classical Sources of Candidate Classes and Objects	9
9.1	Common Sources	10
9.2	Subject Areas	10
9.3	Benefits	10
9.4	Pitfalls	11
10	Behavior Analysis	11
10.1	Core Idea	11
10.2	Responsibilities	11
10.3	Function Points and Outwardly Visible Behavior	11
10.4	Benefits	11
10.5	Common Mistake	12
11	Domain Analysis	12
11.1	Definition	12
11.2	Why It Matters	12
11.3	Typical Steps	12
11.4	Vertical and Horizontal Domain Analysis	12
11.5	Domain Experts	12
12	Use Case Analysis	13
12.1	Definition	13
12.2	Problem It Solves	13
12.3	How Use Cases Drive Object Discovery	13
12.4	Scenario Example: Register for a Course	13
12.5	UML Connection	14
13	CRC Cards	14
13.1	Definition	14
13.2	How CRC Cards Are Used	14
13.3	Dynamic and Static Views	14
13.4	Benefits	15
13.5	Limitations	15

14 Informal English Description	15
14.1 Method	15
14.2 Benefits	15
14.3 Limitations	15
15 Structured Analysis as a Front End	16
15.1 Overview	16
15.2 Candidate Objects from Structured Analysis	16
15.3 Abstraction Analysis	16
15.4 Why This Approach Is Risky	16
15.5 Practical Rule	17
16 Key Abstractions	17
16.1 Definition	17
16.2 Why Key Abstractions Matter	17
16.3 Discovered Versus Invented Abstractions	17
16.4 How to Evaluate a Candidate Abstraction	17
17 Refining Key Abstractions	18
17.1 Placing Abstractions in Hierarchies	18
17.2 Class Promotion	18
17.3 Grainsize Conflict	18
17.4 Cohesion and Coupling Criteria	18
18 Naming Key Abstractions	18
18.1 Why Naming Matters	18
18.2 Naming Guidelines	19
18.3 Naming Anti-Patterns	19
19 Mechanisms	19
19.1 Definition	19
19.2 Mechanisms Versus Classes	19
19.3 Automobile Accelerator Example	20
19.4 Engineering Lesson	20
20 Mechanisms as Strategic Design Decisions	20
20.1 Why Mechanisms Must Be Explicit	20
20.2 Protocol and Mechanism	20
20.3 Mechanism Design Questions	20
21 Mechanisms, Idioms, Patterns, and Frameworks	21
21.1 Pattern Spectrum	21
21.2 Idioms	21
21.3 Frameworks	21
21.4 Design Tradeoff	21

22 Design Pattern: Model–View–Controller	21
22.1 Intent	21
22.2 Problem Solved	21
22.3 Motivation	22
22.4 Structure	22
22.5 Participants	22
22.6 Collaboration	22
22.7 Consequences	22
22.8 Tradeoffs	22
22.9 Real-World Analogy	23
22.10Example Implementation	23
22.11When Not to Use MVC	24
22.12Related Patterns	24
23 Design Pattern: Blackboard	24
23.1 Intent	24
23.2 Problem Solved	24
23.3 Motivation	24
23.4 Structure	24
23.5 Participants	25
23.6 Collaboration	25
23.7 Consequences	25
23.8 Tradeoffs	25
23.9 Real-World Analogy	25
23.10Example Implementation Sketch	25
23.11When Not to Use It	26
23.12Related Patterns	26
24 Other Common Mechanisms	26
25 UML and Architecture Implications	27
25.1 What Diagrams Communicate	27
25.2 How to Interpret Diagrams	27
25.3 Common UML Mistakes	27
25.4 Mermaid Placeholder for a Scenario	28
26 Implementation Considerations	28
26.1 From Analysis Model to Code	28
26.2 Avoiding Anemic Domain Models	28
26.3 Design Tradeoff	29
27 Design Tradeoffs in Classification	29
28 Key Takeaways	30

29 Common Pitfalls	30
29.1 Pitfall 1: Treating Every Noun as a Class	30
29.2 Pitfall 2: Confusing Data Modeling with Object Modeling	30
29.3 Pitfall 3: Overusing Inheritance	30
29.4 Pitfall 4: Ignoring Mechanisms	31
29.5 Pitfall 5: Letting Manager Classes Absorb All Behavior	31
29.6 Pitfall 6: Modeling Too Early at Too Much Detail	31
29.7 Pitfall 7: Ignoring Domain Expert Vocabulary	31
29.8 Pitfall 8: Keeping Structured Analysis Artifacts Too Long	31
29.9 Pitfall 9: Choosing Abstractions Without Lifecycle Clarity	31
29.10 Pitfall 10: Designing for Imaginary Future Variation	31
30 Interview Questions	31
30.1 Question 1	31
30.2 Question 2	31
30.3 Question 3	32
30.4 Question 4	32
30.5 Question 5	32
30.6 Question 6	32
30.7 Question 7	32
30.8 Question 8	32
30.9 Question 9	32
30.10 Question 10	33
30.11 Question 11	33
30.12 Question 12	33
30.13 Question 13	33
30.14 Question 14	33
30.15 Question 15	33
30.16 Question 16	34
30.17 Question 17	34
31 Engineering Insights	34
31.1 Insight 1: Classification Is Design Decision-Making	34
31.2 Insight 2: Scenarios Are Reality Checks	34
31.3 Insight 3: Good Abstractions Have Behavioral Weight	34
31.4 Insight 4: Mechanisms Are Architecture in Motion	34
31.5 Insight 5: Domain Language Is a Design Asset	34
31.6 Insight 6: Classification Should Follow Expected Change	34
31.7 Insight 7: Refactoring Is Part of Classification	34
32 Connections to Other Topics	35
32.1 Connection to Abstraction	35
32.2 Connection to Encapsulation	35
32.3 Connection to Modularity	35
32.4 Connection to Inheritance	35
32.5 Connection to Polymorphism	35
32.6 Connection to SOLID	35
32.7 Connection to Design Patterns	35

32.8	Connection to Architecture	35
32.9	Connection to Testing	36
32.10	Connection to Agile Development	36
33	Practical Applications	36
33.1	Application 1: Student Registration System	36
33.1.1	Candidate Key Abstractions	36
33.1.2	Important Mechanisms	36
33.1.3	Design Insight	36
33.2	Application 2: Online Marketplace	36
33.2.1	Candidate Key Abstractions	36
33.2.2	Important Mechanisms	37
33.2.3	Design Insight	37
33.3	Application 3: Medical Monitoring System	37
33.3.1	Candidate Key Abstractions	37
33.3.2	Important Mechanisms	37
33.3.3	Design Insight	37
33.4	Application 4: Drawing Application	37
33.4.1	Candidate Key Abstractions	37
33.4.2	Important Mechanisms	38
33.4.3	Design Insight	38
34	Summary Cheat Sheet	38

1 Chapter Purpose

This chapter explains how software engineers identify useful classes, objects, abstractions, and mechanisms. In object-oriented analysis and design, **classification** is the act of grouping things that are sufficiently alike for a particular engineering purpose. Classification lets a team discover commonality, reduce duplication, build useful hierarchies, create reusable mechanisms, and design smaller, more coherent architectures.

Core Thesis

There is no perfect class structure and no single correct set of objects. A good object model is a disciplined engineering compromise shaped by the problem domain, stakeholder vocabulary, expected change, quality attributes, implementation constraints, and the designer's judgment.

2 Why Classification Matters in Object-Oriented Design

2.1 Classification as Ordering Knowledge

Classification is the means by which we order knowledge. In software design, it is the process of deciding which things are the same enough to be treated by the same abstraction and which things are different enough to deserve separate abstractions.

- A **class** captures common structure and behavior.
- An **object** represents a concrete instance with identity, state, and behavior.
- A **key abstraction** captures an important concept in the problem domain or solution design.
- A **mechanism** captures how multiple objects collaborate to provide behavior.

2.2 Problem the Concept Solves

Without classification, a system becomes a flat pile of unrelated code, data, and procedures. Proper classification solves several problems:

- **Comprehension:** developers can reason about groups rather than isolated details.
- **Reuse:** common behavior can be factored into shared abstractions.
- **Modularity:** related classes can be packaged together.
- **Extensibility:** variation points can be modeled explicitly.
- **Architecture:** collaborations among abstractions can become stable mechanisms.

2.3 Classification Affects the Entire Design

Classification is not only about naming classes. It affects almost every major design decision.

Design Area	How Classification Helps
Class hierarchy	Identifies generalization and specialization relationships.
Object structure	Identifies aggregation and composition relationships.
Mechanisms	Reveals recurring patterns of object collaboration.
Modularization	Groups classes into modules/packages based on related responsibilities.

Coupling and cohesion	Helps decide which declarations belong together and which should be separated.
Deployment/process allocation	Helps assign processes to processors based on packaging, performance, reliability, and interaction frequency.
Testing	Helps derive test partitions, scenario coverage, and responsibility-based tests.
Maintenance	Makes future changes local because related concepts are represented coherently.

Engineering Interpretation

Classification is a design tool for managing change. When classes reflect real conceptual boundaries, future changes tend to modify fewer classes. When classes are arbitrary, every change leaks across the system.

3 Discovery, Invention, and Sameness

3.1 Discovery

Discovery means recognizing concepts that already exist in the problem domain. These are often the terms used naturally by domain experts.

- In a banking system: **Account, Deposit, Withdrawal, Customer.**
- In a hospital system: **Patient, Medication, Encounter, Nurse.**
- In a railway system: **Train, Dispatcher, Route, Signal.**

3.2 Invention

Invention means creating abstractions that do not necessarily exist in the domain but are necessary for the design or implementation.

- **Repository, Cache, Queue, ScreenManager, DatabaseConnection.**
- Architectural components such as **Controller, Adapter, Gateway, and EventBus.**
- Mechanisms such as validation pipelines, dependency notification, event logging, and model-view synchronization.

Important Distinction

Domain objects are usually discovered. Infrastructure and design objects are often invented. A strong design needs both: discovered abstractions preserve domain meaning, while invented abstractions make the system implementable, testable, and evolvable.

3.3 Classification as Finding Sameness

Classification is fundamentally the search for sameness. In object-oriented design, two things may be considered the same because they share:

- common state or attributes;
- common behavior;

- common responsibilities;
- common role in a collaboration;
- common lifecycle;
- common constraints;
- common variation point;
- common domain meaning.

4 Why Classification Is Difficult

4.1 Boundaries Are Often Fuzzy

A class is supposed to describe objects with clear conceptual boundaries. In practice, boundaries are often ambiguous.

- In a word processor, should the system model **Character**, **Word**, **Sentence**, **Paragraph**, **Selection**, and **Document** as separate classes?
- In speech recognition, where does one word end and the next begin?
- In user-interface design, is a “selection” an object, a property of a document, or a temporary interaction state?

The answer depends on the problem the software must solve. A spell checker cares about words. A layout engine cares about glyphs, lines, paragraphs, and pages. A collaborative editor cares about selections, cursors, operations, and synchronization state.

4.2 There Is No Perfect Classification

Different observers can classify the same thing in different valid ways. The correct classification depends on purpose.

- A car dealership may classify cars by model, color, trim, and inventory status.
- A traffic-control system may classify vehicles by location, speed, lane, and route.
- An insurance system may classify cars by risk, owner, accident history, and policy.

Exam-Ready Statement

A class model is not a mirror of reality. It is a purposeful abstraction of reality designed to support a specific software system.

4.3 Classification Depends on the Observer

Classification is relative to the observer’s goals, vocabulary, and level of abstraction. A programmer may view an object as a primitive. An architect may view an entire subsystem as a primitive. A product owner may classify features by business capability rather than by implementation structure.

4.4 Classification Requires Creative Insight

Classification is not a mechanical recipe. It requires judgment about which similarities matter. Some commonality is accidental and should not be modeled; other commonality reveals a powerful

abstraction.

Common Mistake

Do not create inheritance hierarchies merely because two classes share fields. Shared data is weaker evidence than shared behavior, substitutability, and common meaning.

5 The Incremental and Iterative Nature of Classification

5.1 Classification Evolves During Design

A class structure asserted early in a project is often revised after scenarios, clients, tests, and implementation details reveal new information. This is normal. Object-oriented design is not purely top-down or bottom-up; it moves repeatedly between general concepts and concrete cases.

5.2 Common Refactorings of Classification

Refactoring	Meaning	When It Happens
Derivation	Create a new subclass from an existing class.	A specialized variant appears with additional behavior or constraints.
Factorization	Split a large class into smaller cohesive classes.	A class has too many responsibilities or unrelated reasons to change.
Composition	Unite smaller abstractions into a larger abstraction.	A recurring group of parts behaves as a meaningful whole.
Abstraction	Create a new class to represent previously hidden commonality.	Several classes share behavior, protocol, or conceptual meaning.
Promotion	Move a class upward in the hierarchy to increase sharing.	A concept is more general than originally believed.
Demotion/Specialization	Move behavior downward into specialized subclasses.	A superclass became too broad or imposed irrelevant behavior on subclasses.

5.3 Why Iteration Is Necessary

- Early requirements are incomplete.
- Domain knowledge grows during development.
- Scenarios reveal responsibilities not visible in static noun lists.
- Tests expose awkward interfaces and hidden coupling.
- Users refine their understanding after seeing partial implementations.

Engineering Insight

A good team does not expect the first object model to be perfect. It expects the model to become cleaner as the team learns. The important discipline is to revise abstractions deliberately instead of letting accidental structure harden into architecture.

6 Three Approaches to Classification

Historically, three major theories help explain how humans classify things. Each has direct application to object-oriented analysis and design.

6.1 Overview

Approach	Basis for Sameness	Software Interpretation	Typical Risk
Classical categorization	Necessary and sufficient properties.	Identify classes by attributes and observable behaviors.	Too rigid for fuzzy domains.
Conceptual clustering	Fit to a concept or description.	Group objects by responsibilities, roles, and behavior.	Concepts may be subjective.
Prototype theory	Resemblance to a representative prototype.	Classify by family resemblance to typical examples.	Boundaries may be vague.

6.2 Classical Categorization

6.2.1 Definition

In classical categorization, all entities that share a required property or collection of properties belong to the same category. The properties are treated as necessary and sufficient.

6.2.2 Software Interpretation

A software engineer using classical categorization asks:

- What properties define this class?
- What operations are always meaningful for instances of this class?
- What distinguishes this class from neighboring classes?
- Can membership be decided clearly?

6.2.3 Example

- **RegisteredStudent**: has a student identifier, enrollment status, and academic record.
- **CreditCardPayment**: has card data, authorization result, amount, and merchant.
- **SensorReading**: has timestamp, sensor identifier, value, unit, and validity state.

6.2.4 Benefits

- Produces precise definitions.
- Works well for formal domains such as finance, law, protocols, and data modeling.
- Supports validation and type constraints.
- Makes class membership easier to test.

6.2.5 Limitations

Natural categories are often messy. Many useful concepts cannot be defined by a fixed list of properties. A “chair” may be wooden, plastic, metal, fixed, rolling, reclining, or beanbag-like. A “document” may be a file, web page, generated report, or collaborative data structure.

6.3 Conceptual Clustering

6.3.1 Definition

Conceptual clustering groups entities by first forming a conceptual description and then classifying entities according to how well they fit the description. It is more flexible than classical categorization because membership can depend on approximate fit.

6.3.2 Software Interpretation

Conceptual clustering is useful when a team groups objects by:

- responsibilities;
- roles in scenarios;
- lifecycle behavior;
- collaboration patterns;
- business meaning;
- user-perceived function.

6.3.3 Example: “Reportable Event”

A monitoring system might classify several things as **ReportableEvent**: threshold violations, manual alerts, device failures, and abnormal trends. These do not necessarily share identical data fields, but they fit the concept of “something that should be recorded, displayed, escalated, or audited.”

6.3.4 Benefits

- Handles fuzzy business concepts.
- Encourages behavior-centered modeling.
- Helps identify abstractions that classical property lists miss.
- Supports responsibility-driven design.

6.3.5 Tradeoff

The flexibility of conceptual clustering can produce subjective models. Teams should validate clusters using scenarios, tests, domain expert review, and change-impact analysis.

6.4 Prototype Theory

6.4.1 Definition

Prototype theory classifies objects by comparing them to a representative prototype. An object belongs to a category if it sufficiently resembles the prototype in relevant ways.

6.4.2 Software Interpretation

Prototype theory is useful when categories have no fixed boundary but have recognizable examples.

- A “fraudulent transaction” may resemble known suspicious patterns.
- A “premium customer” may resemble a typical high-value customer but not satisfy one rigid rule.
- A “document” in a content platform may be text, scanned PDF, wiki page, or generated export.

6.4.3 Interactional Properties

Prototype theory emphasizes properties that matter because of how users interact with the object. A beanbag chair and a barber chair may not share a strict structural definition, but both support the interaction “sit in it.”

6.4.4 Software Engineering Use

Prototype theory supports:

- UX-centered classification;
- machine learning feature interpretation;
- heuristic business rules;
- flexible taxonomies;
- plugin systems where new variants resemble known examples.

Design Tradeoff

Prototype-based classification can model real-world fuzziness, but it may be difficult to encode as simple type hierarchies. In implementation, it often appears as rules, policies, scoring functions, tags, strategies, or classifiers rather than inheritance alone.

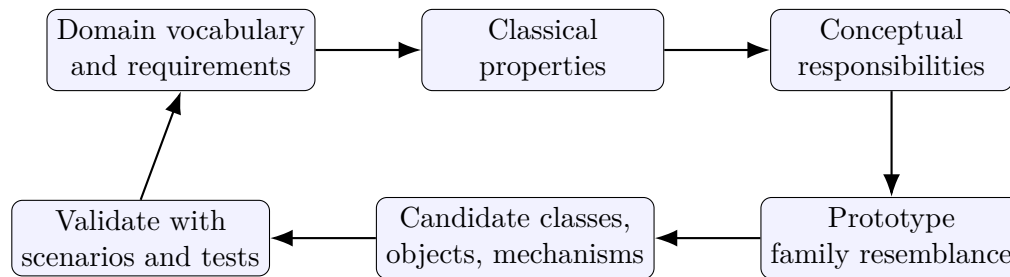
7 Applying Classification Theory to Object-Oriented Analysis

7.1 Recommended Order of Use

A practical object-oriented analysis process often applies the three theories in the following order:

1. **Start with properties and domain vocabulary.** Identify obvious domain classes.
2. **Refine by concepts and behavior.** Group objects by responsibilities and collaborations.
3. **Use prototype association for fuzzy cases.** Model categories that are recognized by resemblance rather than strict definition.

7.2 Conceptual Workflow



7.3 Engineering Rule of Thumb

Classification Heuristic

If a candidate class is hard to name, hard to assign responsibilities to, hard to test, and hard to place in scenarios, it is probably not yet a clean abstraction.

8 Object-Oriented Analysis

8.1 Analysis Versus Design

The boundary between analysis and design is fuzzy, but their emphasis is different.

Activity	Primary Focus	Typical Output
Analysis	Understand and model the problem. Discover vocabulary used by users and domain experts.	Domain concepts, use cases, scenarios, responsibilities, constraints.
Design	Invent the solution structure. Decide how objects collaborate and how implementation concerns are handled.	Classes, interfaces, mechanisms, packages, frameworks, patterns, deployment decisions.

8.2 Why Naive Analysis Fails

A naive approach often begins by listing nouns from requirements and turning each noun into a class. This fails because:

- language is ambiguous;
- important responsibilities may be verbs or relationships rather than nouns;
- some nouns are attributes, values, roles, or temporary states;
- some important design abstractions are not mentioned by users;
- requirements often describe the current workflow rather than the best software model.

9 Classical Sources of Candidate Classes and Objects

Classical analysis identifies objects from the concrete and conceptual things mentioned in the problem domain.

9.1 Common Sources

Source	Meaning	Examples
Tangible things	Physical or digital things that matter to the system.	Car, sensor, file, package.
Roles	A function played by a person, system, or object.	Teacher, dispatcher, reviewer.
Events	Something that happens and may need to be recorded or handled.	Landing, request, interrupt.
Interactions	Relationship-like events involving multiple participants.	Loan, meeting, transaction.
People	Human actors or domain participants.	Patient, customer, clerk.
Places	Locations relevant to rules or behavior.	Warehouse, clinic, airport gate.
Organizations	Groups with mission, resources, and identity.	Department, bank, airline.
Concepts	Intangible business ideas.	Policy, permission, contract.
External systems	Neighboring systems with which the application interacts.	Payment gateway, identity provider.
Devices	Hardware or embedded interfaces.	Scanner, pump, GPS receiver.
Remembered events	Historical facts the system must preserve.	Login attempt, shipment delay.
Locations	Physical or logical places important to operation.	Server region, store aisle.
Organizational units	Groups to which users or resources belong.	Team, division, class section.

9.2 Subject Areas

A **subject area** is a logical group of related classes that supports a higher-level system function. In modern terms, subject areas often become packages, bounded contexts, services, modules, or feature areas.

- Student registration system: **Enrollment, Billing, Advising, Catalog.**
- E-commerce system: **Catalog, Cart, Checkout, Fulfillment.**
- Hospital system: **PatientRecords, Scheduling, Medication, Billing.**

9.3 Benefits

- Easy to teach and apply.
- Starts from domain vocabulary.
- Works well for information systems and business domains.
- Produces candidate objects quickly.

9.4 Pitfalls

- Produces too many passive data classes if behavior is ignored.
- May confuse attributes with classes.
- May miss invented design abstractions such as repositories, services, policies, and coordinators.
- May overemphasize nouns and underemphasize collaborations.

10 Behavior Analysis

10.1 Core Idea

Behavior analysis identifies classes and objects by examining what the system does. Instead of asking only “What things exist?”, it asks:

- What behavior must the system provide?
- Who initiates the behavior?
- Who participates in the behavior?
- Which objects know the necessary information?
- Which objects should be responsible for each action?

10.2 Responsibilities

A **responsibility** is the knowledge an object maintains or the action it can perform. Responsibilities express the purpose of an object and its role in the system.

Responsibility Type	Meaning	Example
Knowing	The object maintains or can derive information.	Account knows its balance.
Doing	The object performs an action.	Invoice calculates total.
Coordinating	The object delegates work to collaborators.	CheckoutController coordinates payment and shipping.
Enforcing	The object protects an invariant or rule.	EnrollmentPolicy checks prerequisites.
Notifying	The object communicates changes.	EventBus publishes events.

10.3 Function Points and Outwardly Visible Behavior

A **function point** can be understood as an outwardly visible, testable behavior of the system. Examples include an output, inquiry, input, stored file, or external interface. Function points help identify system behavior before assigning that behavior to classes.

10.4 Benefits

- Encourages active objects rather than passive records.
- Reveals collaborations and message flow.
- Supports testable design because responsibilities map naturally to tests.

- Helps assign behavior to the object with the necessary knowledge.

10.5 Common Mistake

Do not create one manager class that performs every system function. That centralizes behavior, increases coupling, and turns domain objects into anemic data containers.

11 Domain Analysis

11.1 Definition

Domain analysis identifies the objects, operations, and relationships that domain experts perceive to be important in a family of related applications.

11.2 Why It Matters

Most software systems are not entirely unique. Patient record systems, compilers, trading systems, reservation systems, embedded controllers, and reporting systems often reuse concepts and mechanisms from earlier systems. Domain analysis prevents teams from rediscovering known abstractions through trial and error.

11.3 Typical Steps

1. Construct a preliminary generic model with help from domain experts.
2. Examine existing systems in the same domain.
3. Represent existing systems in a common format.
4. Identify similarities and differences among systems.
5. Refine the generic model to accommodate important variations.

11.4 Vertical and Horizontal Domain Analysis

Type	Meaning	Example
Vertical domain analysis	Compares similar systems in the same application domain.	Compare multiple patient-monitoring systems to identify common abstractions such as sensor, alert, threshold, and clinician.
Horizontal domain analysis	Compares related parts within the same system or across many domains.	Analyze many report types in one accounting system to build a reusable reporting mechanism.

11.5 Domain Experts

A **domain expert** is someone deeply familiar with the problem domain, usually not a software developer. Examples include nurses, dispatchers, traders, pilots, warehouse workers, financial analysts, and compliance officers.

Practical Advice

A short conversation with a domain expert can prevent weeks of bad design. Domain analysis is often most effective when done repeatedly in small focused sessions rather than as one monolithic upfront activity.

12 Use Case Analysis

12.1 Definition

A **use case** is a behaviorally related sequence of transactions performed by an actor in dialogue with the system to produce measurable value for that actor.

12.2 Problem It Solves

Use case analysis gives structure to object discovery. It prevents analysis from depending only on intuition by forcing the team to examine concrete scenarios.

12.3 How Use Cases Drive Object Discovery

1. Enumerate fundamental scenarios early in requirements analysis.
2. For each scenario, identify participating actors and objects.
3. Identify each object's responsibilities.
4. Identify collaborations: which object asks which other object to do what.
5. Expand scenarios to include exceptional flows and secondary behaviors.
6. Revise abstractions when scenarios reveal missing or misplaced responsibilities.
7. Use scenarios as the basis for system tests.

12.4 Scenario Example: Register for a Course

Scenario Element	Example
Actor	Student.
Goal	Enroll in an available course section.
Candidate objects	Student , Course , Section , Enrollment , PrerequisitePolicy , Schedule .
Responsibilities	Student requests enrollment; Section knows capacity; PrerequisitePolicy validates eligibility; Enrollment records the successful registration.
Collaborations	Student interface calls enrollment service; service checks policies; section reserves seat; enrollment record is created.
Exceptions	Section full, prerequisite missing, time conflict, hold on account.
Tests	Successful enrollment, duplicate enrollment rejection, capacity boundary, prerequisite failure.

12.5 UML Connection

- **Use case diagrams** show actors and system goals.
- **Sequence diagrams** show object collaboration over time for a scenario.
- **Communication diagrams** show links and messages among collaborating objects.
- **Class diagrams** stabilize the participating abstractions and their relationships.
- **State machine diagrams** clarify lifecycle-heavy objects found during scenarios.

13 CRC Cards

13.1 Definition

CRC stands for **Class–Responsibilities–Collaborators**. A CRC card is a simple index-card representation of one candidate class.

Class: Enrollment	
Responsibilities Validate request Record registration Cancel enrollment	Collaborators Student Section PrerequisitePolicy Schedule

13.2 How CRC Cards Are Used

1. Create one card for each candidate class.
2. Write responsibilities on one side and collaborators on the other.
3. Walk through a scenario.
4. Assign responsibilities to cards.
5. Split, merge, or rename cards when responsibilities are awkward.
6. Arrange cards spatially to visualize collaborations or hierarchies.

13.3 Dynamic and Static Views

View	What the Cards Show	UML Connection
Dynamic semantics	Flow of messages among prototypical instances during a scenario.	Sequence or communication diagram.
Static semantics	Generalization, specialization, aggregation, and association relationships.	Class diagram or package diagram.

13.4 Benefits

- Lightweight and easy to revise.
- Encourages team discussion.
- Keeps attention on responsibilities instead of only data.
- Makes collaboration visible before code is written.

13.5 Limitations

CRC cards are informal. They are excellent for brainstorming and responsibility allocation but should eventually be supported by more precise models, tests, and code.

14 Informal English Description

14.1 Method

Write a natural-language description of the problem. Underline nouns as candidate objects and verbs as candidate operations.

Listing 1: Example noun/verb analysis

```
A student requests enrollment in a course section.  
The system checks prerequisites and seat availability.  
If the request is valid, the system creates an enrollment record.
```

Candidate objects: Student, Enrollment, CourseSection, System, Prerequisite, SeatAvailability, EnrollmentRecord.

Candidate operations: requestEnrollment, checkPrerequisites, checkSeatAvailability, createEnrollmentRecord.

14.2 Benefits

- Simple and accessible.
- Forces use of domain vocabulary.
- Useful for small problems and early brainstorming.

14.3 Limitations

- Does not scale to complex systems.
- Depends heavily on writing quality.
- Language is imprecise: nouns can become verbs and verbs can become nouns.
- Can produce too many candidate classes.
- May bias the model toward either objects or operations based on phrasing.

15 Structured Analysis as a Front End

15.1 Overview

Some organizations attempt to use structured analysis artifacts, especially data flow diagrams, as input to object-oriented design. This can be tempting when teams already know structured analysis or have CASE tools supporting it.

15.2 Candidate Objects from Structured Analysis

Structured Artifact	Possible Object-Oriented Interpretation
Data dictionary	Essential data elements may point to candidate objects. Ask what noun each data element describes.
Context diagram	External entities and system boundaries suggest actors, neighboring systems, and interface objects.
External entities	Candidate objects or actors outside the system boundary.
Data stores	Candidate persistent objects, repositories, or aggregates.
Control stores	Candidate state-holding or policy objects.
Control transformations	Candidate controller, coordinator, or state-transition objects.
Data flows	Candidate classes or information objects.
Control flows	Candidate event, command, or signal classes.
Data transformations	Usually operations on existing objects, or behavior assigned to an invented agent object.

15.3 Abstraction Analysis

Abstraction analysis examines inputs and outputs and follows them inward to identify a central entity or abstract model of what the system does. Supporting entities are found by tracing flows around the central entity.

15.4 Why This Approach Is Risky

Structured Analysis Warning

Structured analysis and object-oriented design are based on different decompositions. Structured analysis tends to emphasize functions and data flows; object-oriented design emphasizes collaborating objects with responsibilities.

Risks include:

- Analysts may write data flow diagrams that already contain design assumptions.
- The resulting model may be biased toward algorithmic decomposition.
- Developers may fall back into procedural design.
- Object collaboration may be treated as an afterthought.

15.5 Practical Rule

Use structured analysis artifacts only while they help understand the essential problem. Stop relying on them when they begin to describe a procedural solution rather than the problem itself.

16 Key Abstractions

16.1 Definition

A **key abstraction** is a class or object that forms part of the vocabulary of the problem domain or solution design. Key abstractions define the conceptual boundary of the system: they identify what is relevant and suppress what is irrelevant.

16.2 Why Key Abstractions Matter

- They give the design a shared vocabulary.
- They reduce conceptual noise.
- They become anchors for responsibilities and collaborations.
- They guide package/module boundaries.
- They make the system explainable to domain experts and developers.

16.3 Discovered Versus Invented Abstractions

Type	Description	Examples
Discovered abstraction	Comes from domain expert vocabulary or real domain structure.	Account, deposit, withdrawal, patient, route, invoice.
Invented abstraction	Created by designers to support implementation, coordination, reuse, or infrastructure.	Database, queue, repository, screen manager, validation pipeline, event bus.

16.4 How to Evaluate a Candidate Abstraction

A candidate abstraction is not automatically a good class. Ask:

- How are objects of this class created?
- Can objects of this class be copied? Should they be copied?
- How are objects destroyed or retired?
- What operations are meaningful?
- What invariants must always hold?
- What responsibilities does it own?
- Which scenarios require it?
- Which collaborators does it need?
- Is the abstraction cohesive?
- Is it loosely coupled to unrelated concepts?

- Does its name make sense to domain experts or developers?

Clean Abstraction Test

If there are no good answers for creation, lifecycle, operations, invariants, and responsibility, the concept is probably not clean enough to code yet.

17 Refining Key Abstractions

17.1 Placing Abstractions in Hierarchies

New abstractions must be placed in existing class and object hierarchies. This is rarely purely top-down or bottom-up. Designers often create several classes, notice commonality, factor that commonality upward, and then refine subclasses.

17.2 Class Promotion

Class promotion moves a concept upward in a hierarchy when it is more general than first thought. This increases sharing but can also make the superclass too broad if done carelessly.

17.3 Grainsize Conflict

A **grainsize conflict** occurs when a class is too general or too coarse, creating a semantic gap for subclasses. Subclasses must then inherit behavior that does not fit them or override too much.

17.4 Cohesion and Coupling Criteria

Criterion	Good Sign	Bad Sign
Cohesion	Responsibilities naturally belong together.	Class mixes unrelated responsibilities.
Coupling	Collaborations are explicit and minimal.	Class knows details of many unrelated classes.
Level of abstraction	Class is neither too broad nor too narrow.	Class is either vague or trivial.
Substitutability	Subclass can be used where superclass is expected.	Subclass violates superclass meaning.
Lifecycle	Creation, mutation, and disposal are clear.	Object exists only because implementation needed a place to put data.

18 Naming Key Abstractions

18.1 Why Naming Matters

Names carry semantics. A good name communicates what an abstraction is, what it is not, and how it should be used. Poor names hide design mistakes and make maintenance harder.

18.2 Naming Guidelines

Element	Naming Guideline	Examples
Objects	Proper noun phrases or meaningful instance names.	<code>theSensor</code> , <code>currentShape</code> .
Classes	Common noun phrases.	<code>Sensor</code> , <code>Shape</code> , <code>Enrollment</code> .
Modifier operations	Active verb phrases.	<code>draw</code> , <code>moveLeft</code> , <code>reserveSeat</code> .
Selector/query operations	Query-like names or “to be” forms.	<code>extentOf</code> , <code>isOpen</code> , <code>hasPrerequisite</code> .
Domain concepts	Use domain expert vocabulary when possible.	<code>Claim</code> , <code>Policy</code> , <code>MedicationOrder</code> .
Style	Be internally consistent with capitalization and separators.	<code>camelCase</code> or <code>snake_case</code> , not random mixing.

18.3 Naming Anti-Patterns

- **DataManager**: vague and often too powerful.
- **Helper**: unclear responsibility.
- **Processor**: procedural smell unless the domain truly has a processor.
- **Info**: usually an anemic data holder.
- **Util**: often hides missing abstractions.

19 Mechanisms

19.1 Definition

A **mechanism** is a structure by which objects collaborate to provide higher-level behavior. A class describes how one kind of object behaves; a mechanism describes how many objects cooperate.

Core Definition

Mechanisms are the means by which objects collaborate to provide behavior that no single object should own alone.

19.2 Mechanisms Versus Classes

Aspect	Class	Mechanism
Scope	Individual abstraction.	Collaboration among abstractions.
Design level	Tactical design decision.	Strategic design decision.
Question answered	What does this object know and do?	How do these objects cooperate?
Typical model	Class diagram, interface, state machine.	Sequence diagram, communication diagram, architecture pattern.
Reuse level	Reuse of type or implementation.	Reuse of collaboration structure.

Example	Window, View, Model.	Model–View–Controller drawing/update mechanism.	draw-
---------	----------------------	--	-------

19.3 Automobile Accelerator Example

Requirement: pressing the accelerator should cause the engine to run faster; releasing it should cause the engine to run slower.

Possible mechanisms:

- mechanical linkage from accelerator to fuel injectors;
- electronic pressure sensor connected to a computer controlling fuel injection;
- gravity-based fuel flow regulated by a physical clip.

All can satisfy the same external requirement, but they differ in cost, reliability, manufacturability, maintainability, safety, and performance.

19.4 Engineering Lesson

Requirements describe externally visible behavior. Mechanisms are design choices about how that behavior is achieved. Different mechanisms may satisfy the same requirement but produce very different architectures.

20 Mechanisms as Strategic Design Decisions

20.1 Why Mechanisms Must Be Explicit

If mechanisms are not intentionally designed, the system becomes a mob of uncoordinated objects. Each object may be locally reasonable, but global behavior becomes accidental.

20.2 Protocol and Mechanism

The protocol of an individual class eventually includes all operations needed to participate in the mechanisms associated with its instances. Therefore, object interfaces should be shaped by real collaborations, not by imagined data access needs.

20.3 Mechanism Design Questions

- Which objects participate?
- Which object initiates the collaboration?
- Which object owns the key decision?
- What messages are exchanged?
- What should be synchronous versus asynchronous?
- What failures can occur?
- What invariants must the mechanism preserve?
- Which parts should be replaceable?
- Does the mechanism reduce or increase coupling?

21 Mechanisms, Idioms, Patterns, and Frameworks

21.1 Pattern Spectrum

Level	Meaning	Example
Idiom	Language- or culture-specific convention for a recurring low-level task.	Resource handling idioms, naming conventions, iterator usage.
Mechanism	Collaboration among objects to produce behavior.	Dependency notification, event logging, MVC drawing update.
Design pattern	Named reusable solution to a recurring design problem.	Observer, Strategy, MVC, Command.
Framework	Reusable collection of classes and mechanisms for a domain.	JUnit, web frameworks, GUI frameworks, application platforms.

21.2 Idioms

An **idiom** is an accepted programming-language or application-culture convention. Idioms are low-level patterns. They improve communication among developers who know the language culture.

21.3 Frameworks

A **framework** is a reusable collection of classes that provides services for a particular domain. Frameworks represent reuse in the large because they provide multiple abstractions and mechanisms that clients adapt or extend.

21.4 Design Tradeoff

- Idioms are lightweight but language-specific.
- Mechanisms are reusable but require design discipline.
- Patterns are communicative but can be overused.
- Frameworks provide large-scale reuse but impose architecture and learning costs.

22 Design Pattern: Model–View–Controller

The chapter uses graphical user-interface drawing as an example of a mechanism. This mechanism is commonly recognized as the Model–View–Controller pattern.

22.1 Intent

Separate domain state, user-interface presentation, and input/control logic so that each can change independently.

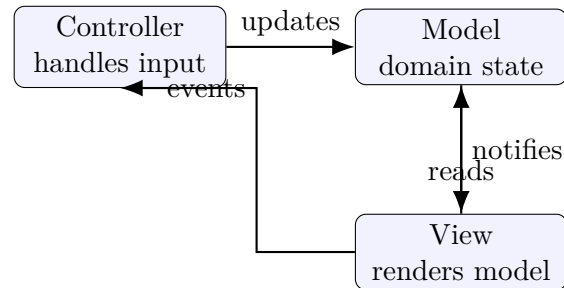
22.2 Problem Solved

Naive UI code often mixes data, rendering, and user interaction in the same class. This causes high coupling and makes it difficult to test business logic, support multiple views, or change presentation behavior.

22.3 Motivation

A model should not know every possible way it can be displayed. A view should render data but should not own domain rules. A controller should translate user intent into operations without becoming the domain model.

22.4 Structure



22.5 Participants

- **Model:** owns domain data and rules.
- **View:** displays model state.
- **Controller:** handles user input and invokes model operations.
- **Client/Application:** wires model, view, and controller together.

22.6 Collaboration

A typical drawing collaboration is:

1. A client asks a window or view to draw.
2. The window tells subviews to draw themselves.
3. Each view reads or asks its model for displayable state.
4. The model remains decoupled from rendering details.

22.7 Consequences

- Multiple views can display the same model.
- UI changes do not necessarily change domain logic.
- Model logic becomes easier to test.
- More objects and message paths are introduced.

22.8 Tradeoffs

- **Benefit:** strong separation of concerns.
- **Cost:** more indirection and coordination.
- **Risk:** controllers can become bloated if business logic is placed there.

22.9 Real-World Analogy

A restaurant separates recipe knowledge, menu presentation, and order taking. The kitchen owns food preparation rules, the menu displays options, and the waiter/controller interprets customer requests.

22.10 Example Implementation

Listing 2: Minimal MVC-style collaboration

```
final class CourseSectionModel {
    private int capacity;
    private int enrolled;

    boolean hasSeat() {
        return enrolled < capacity;
    }

    void enroll() {
        if (!hasSeat()) {
            throw new IllegalStateException("Section is full");
        }
        enrolled++;
    }

    int seatsRemaining() {
        return capacity - enrolled;
    }
}

final class CourseSectionView {
    void render(CourseSectionModel model) {
        System.out.println("Seats remaining: " + model.seatsRemaining());
    }
}

final class EnrollmentController {
    private final CourseSectionModel model;
    private final CourseSectionView view;

    EnrollmentController(CourseSectionModel model, CourseSectionView view)
    {
        this.model = model;
        this.view = view;
    }

    void handleEnrollClick() {
        model.enroll();           // domain operation
        view.render(model);      // presentation update
    }
}
```

22.11 When Not to Use MVC

- Tiny scripts or simple pages where the separation adds more complexity than value.
- Domains where rendering and state are inseparable and unlikely to change.
- Teams that use MVC labels but still place all logic in controllers.

22.12 Related Patterns

- **Observer:** often used for model-to-view notification.
- **Strategy:** can vary rendering or input behavior.
- **Command:** can represent user actions.
- **Presentation Model / MVVM:** related UI separation patterns.

23 Design Pattern: Blackboard

The chapter also mentions the blackboard mechanism, common in artificial intelligence and reasoning systems.

23.1 Intent

Allow independent knowledge sources or agents to cooperate by reading and writing a shared problem-solving workspace.

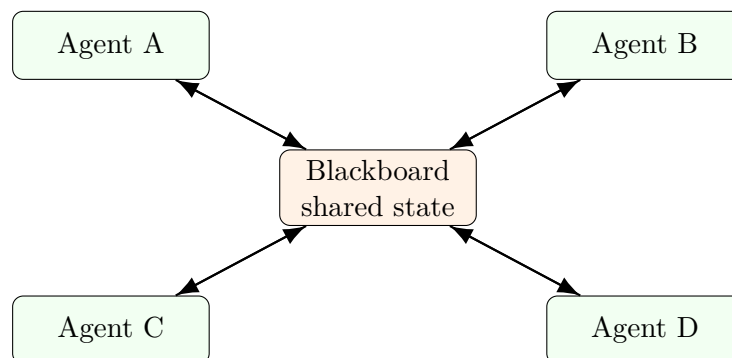
23.2 Problem Solved

Some problems cannot be solved cleanly by one centralized algorithm. Different specialists may contribute partial solutions when relevant information appears.

23.3 Motivation

In speech recognition, planning, diagnosis, or complex reasoning, one subsystem may detect a partial clue, another may refine it, and another may combine it with context. A blackboard allows progress without requiring one controller to know every possible reasoning path.

23.4 Structure



23.5 Participants

- **Blackboard:** shared repository of partial solutions and state.
- **Knowledge sources/agents:** independent contributors that inspect and update the blackboard.
- **Control policy:** optional strategy deciding which agent acts next.

23.6 Collaboration

Agents monitor the blackboard. A change may trigger an agent to explore a new path, add information, revise a hypothesis, or signal completion.

23.7 Consequences

- Supports opportunistic and incremental problem solving.
- Decouples agents from direct knowledge of each other.
- Makes global state and coordination policy critical.
- Can be hard to debug because behavior emerges from many agents.

23.8 Tradeoffs

- **Benefit:** flexible collaboration among specialized components.
- **Cost:** shared state can become a bottleneck or source of hidden coupling.
- **Risk:** without clear rules, agents can interfere with one another.

23.9 Real-World Analogy

A hospital emergency board lists patient status, tasks, and updates. Doctors, nurses, technicians, and administrators each act when information relevant to them appears.

23.10 Example Implementation Sketch

Listing 3: Simplified blackboard-style collaboration

```
interface Agent {
    boolean canActOn(Blackboard board);
    void act(Blackboard board);
}

final class Blackboard {
    private final Map<String, Object> facts = new HashMap<>();

    void put(String key, Object value) {
        facts.put(key, value);
    }

    Optional<Object> get(String key) {
        return Optional.ofNullable(facts.get(key));
    }
}
```

```

final class ReasoningLoop {
    private final List<Agent> agents;

    ReasoningLoop(List<Agent> agents) {
        this.agents = agents;
    }

    void run(Blackboard board) {
        boolean changed;
        do {
            changed = false;
            for (Agent agent : agents) {
                if (agent.canActOn(board)) {
                    agent.act(board);
                    changed = true;
                }
            }
        } while (changed);
    }
}

```

23.11 When Not to Use It

- When a simple pipeline or direct workflow is sufficient.
- When strong transactional consistency is required and shared state would be dangerous.
- When agent interactions cannot be monitored, tested, or bounded.

23.12 Related Patterns

- **Observer:** agents may subscribe to blackboard changes.
- **Mediator:** centralizes communication but usually with more explicit coordination.
- **Event-driven architecture:** replaces shared board updates with event streams.
- **Pipes and filters:** more linear alternative.

24 Other Common Mechanisms

Mechanism	Purpose	Modern Example
Dependency notification	Notify dependents when an object changes.	Observer pattern, reactive streams, UI binding.
Time association	Relate objects or events across time.	Audit timelines, event sourcing, temporal validity.
Event logging	Preserve meaningful events for later analysis.	Application logs, audit trails, observability pipelines.
Broadcasting	Send a message to multiple receivers.	Pub/sub, event bus, notification service.

Dispatching	Decide which program, task, or handler runs next.	OS scheduler, message dispatcher, request router.
Validation pipeline	Apply a sequence of validation rules.	Form validation, policy engines, API request validation.

25 UML and Architecture Implications

25.1 What Diagrams Communicate

Diagram	What It Communicates	Use During Classification
Use case diagram	Actors and goals.	Establish scenario boundaries and user value.
Class diagram	Classes, attributes, operations, and relationships.	Stabilize discovered abstractions.
Object diagram	Snapshot of object instances and links.	Test whether class relationships make sense for concrete cases.
Sequence diagram	Time-ordered messages among objects.	Validate mechanisms and responsibility assignment.
Communication diagram	Object links and message paths.	See collaboration structure without strict time focus.
State machine diagram	States and transitions of a lifecycle-heavy object.	Refine classes with complex behavior.
Package diagram	Groups of related classes.	Reflect subject areas and modular boundaries.
Deployment diagram	Processes, nodes, and allocation.	Connect classification to performance, reliability, and distribution.

25.2 How to Interpret Diagrams

A good diagram should answer a design question. For example:

- A class diagram should answer: “What are the major abstractions and relationships?”
- A sequence diagram should answer: “How do objects collaborate to satisfy this scenario?”
- A state machine should answer: “What lifecycle constraints govern this object?”
- A package diagram should answer: “Which abstractions change together and belong together?”

25.3 Common UML Mistakes

- Drawing classes before understanding scenarios.
- Treating every noun as a class.
- Modeling database tables instead of domain behavior.
- Using inheritance where composition or delegation is better.
- Omitting multiplicities, ownership, and lifecycle constraints.

- Creating diagrams that are too detailed to communicate.
- Forgetting that diagrams are tools, not deliverables with value by themselves.

25.4 Mermaid Placeholder for a Scenario

The following textual diagram can be pasted into Mermaid-enabled tooling.

Listing 4: Mermaid sequence diagram placeholder

```
sequenceDiagram
    actor Student
    participant UI as Registration UI
    participant Service as Enrollment Service
    participant Policy as Prerequisite Policy
    participant Section as Course Section
    participant Enrollment as Enrollment Record

    Student->>UI: request enrollment
    UI->>Service: enroll(student, section)
    Service->>Policy: verify(student, section)
    Service->>Section: reserveSeat()
    Service->>Enrollment: create(student, section)
    Service-->>UI: confirmation
```

26 Implementation Considerations

26.1 From Analysis Model to Code

A candidate analysis class should not be translated blindly into code. Implementation requires additional decisions:

- persistence strategy;
- concurrency strategy;
- error handling;
- validation boundaries;
- interfaces and dependency inversion;
- framework integration;
- testing approach;
- performance and deployment constraints.

26.2 Avoiding Anemic Domain Models

An anemic domain model has classes that hold data but little behavior. It usually happens when responsibilities are assigned to service classes instead of domain objects.

Listing 5: Anemic model smell

```
final class Account {
    BigDecimal balance; // data only
}
```

```
final class AccountService {
    void withdraw(Account account, BigDecimal amount) {
        if (account.balance.compareTo(amount) < 0) {
            throw new IllegalArgumentException("Insufficient funds");
        }
        account.balance = account.balance.subtract(amount);
    }
}
```

A more object-oriented model assigns responsibility to the object that owns the invariant.

Listing 6: Responsibility assigned to the domain object

```
final class Account {
    private BigDecimal balance;

    void withdraw(BigDecimal amount) {
        if (balance.compareTo(amount) < 0) {
            throw new IllegalArgumentException("Insufficient funds");
        }
        balance = balance.subtract(amount);
    }

    BigDecimal balance() {
        return balance;
    }
}
```

26.3 Design Tradeoff

Not every rule belongs inside an entity. Cross-entity rules, external integrations, and orchestration logic may belong in services, policies, domain events, or mechanisms. The key is to avoid both extremes:

- **God service:** all behavior centralized in one procedural controller.
- **Overloaded entity:** one domain object knows too much about unrelated workflows.

27 Design Tradeoffs in Classification

Tradeoff	Benefit	Risk
More classes	Fine-grained responsibilities and easier testing.	Too much indirection and navigation overhead.
Fewer classes	Simpler initial model.	Large classes, low cohesion, hidden responsibilities.
Inheritance	Shared protocol and polymorphism.	Fragile hierarchies and semantic mismatch.
Composition	Flexible runtime assembly.	More explicit delegation and object wiring.

Domain purity	Model matches business language.	Infrastructure concerns may become awkward.
Technical pragmatism	Easier integration with frameworks and storage.	Domain meaning may be diluted.
Upfront modeling	Early shared understanding.	Risk of speculative design.
Evolutionary modeling	Adapts to learning.	Requires refactoring discipline.

28 Key Takeaways

1. Classification is central to object-oriented analysis and design because it identifies useful sameness.
2. Class and object identification involves both discovery of domain concepts and invention of design abstractions.
3. There is no perfect classification; a class model is judged by how well it serves the system's purpose.
4. Classification is iterative. Good models emerge through scenarios, domain learning, refactoring, and implementation feedback.
5. Classical categorization groups by properties; conceptual clustering groups by concepts and responsibilities; prototype theory groups by resemblance to representative examples.
6. Use cases and scenarios are powerful because they reveal responsibilities and collaborations.
7. CRC cards are lightweight tools for exploring classes, responsibilities, and collaborators.
8. Structured analysis artifacts can help, but they can also bias a team toward procedural decomposition.
9. Key abstractions define the vocabulary and boundary of the system.
10. Mechanisms define strategic collaboration structures among objects.
11. The best object-oriented systems combine clean abstractions with carefully engineered mechanisms.

29 Common Pitfalls

29.1 Pitfall 1: Treating Every Noun as a Class

Noun extraction is only a brainstorming technique. Some nouns are attributes, values, roles, events, or implementation details.

29.2 Pitfall 2: Confusing Data Modeling with Object Modeling

Database tables describe storage. Objects should also describe behavior, responsibility, and collaboration.

29.3 Pitfall 3: Overusing Inheritance

Inheritance should represent meaningful substitutability and shared behavior, not merely duplicated fields.

29.4 Pitfall 4: Ignoring Mechanisms

A system with many well-named classes can still fail if object collaboration is accidental or unclear.

29.5 Pitfall 5: Letting Manager Classes Absorb All Behavior

Classes named `Manager`, `Processor`, or `Service` often hide missing domain responsibilities.

29.6 Pitfall 6: Modeling Too Early at Too Much Detail

Prematurely detailed class diagrams can freeze bad assumptions before scenarios have been explored.

29.7 Pitfall 7: Ignoring Domain Expert Vocabulary

When software terms do not match domain language, communication cost rises and domain rules are misunderstood.

29.8 Pitfall 8: Keeping Structured Analysis Artifacts Too Long

Data flow diagrams can help understand the problem, but they should not force an object-oriented design into an algorithmic structure.

29.9 Pitfall 9: Choosing Abstractions Without Lifecycle Clarity

If creation, copying, mutation, and destruction are unclear, the abstraction may not be clean.

29.10 Pitfall 10: Designing for Imaginary Future Variation

Do not invent elaborate hierarchies or frameworks without evidence from scenarios, domain analysis, or likely change.

30 Interview Questions

30.1 Question 1

Question: Why is classification important in object-oriented design?

Concise Answer: Classification identifies meaningful sameness among concepts, which allows designers to create classes, hierarchies, modules, and mechanisms. It reduces complexity by grouping related behavior and structure, but it must be guided by the system's purpose because there is no universally correct class model.

30.2 Question 2

Question: What is the difference between discovering and inventing abstractions?

Concise Answer: Discovery identifies abstractions already present in the problem domain, usually reflected in domain expert vocabulary. Invention creates new abstractions needed for the solution, such as repositories, queues, controllers, or event buses. Good designs usually require both domain meaning and implementation support.

30.3 Question 3

Question: Why is there no perfect class structure?

Concise Answer: A class structure is a purposeful model, not a complete representation of reality. Different systems classify the same real-world concepts differently depending on required behavior, constraints, users, performance needs, and expected change. A good model is one that supports the intended design tradeoffs.

30.4 Question 4

Question: Compare classical categorization, conceptual clustering, and prototype theory.

Concise Answer: Classical categorization groups objects by necessary and sufficient properties. Conceptual clustering groups objects by fit to a conceptual description or responsibility. Prototype theory groups objects by resemblance to a representative example. Classical categorization is precise, conceptual clustering is behavior-friendly, and prototype theory handles fuzzy real-world categories.

30.5 Question 5

Question: How do use cases help identify classes and objects?

Concise Answer: Use cases describe concrete actor goals and system interactions. Walking through scenarios reveals participating objects, responsibilities, collaborations, exceptional cases, and test cases. This prevents the model from being a static noun list and grounds the design in required behavior.

30.6 Question 6

Question: What are CRC cards and why are they useful?

Concise Answer: CRC cards represent candidate classes with their responsibilities and collaborators. They are useful because they make responsibility assignment and object collaboration tangible, lightweight, and easy to revise during scenario walkthroughs.

30.7 Question 7

Question: What is a key abstraction?

Concise Answer: A key abstraction is a class or object central to the vocabulary of the problem domain or solution. It defines what is relevant to the system and provides a stable concept around which responsibilities, collaborations, and modules can be organized.

30.8 Question 8

Question: How do you evaluate whether a candidate class is a good abstraction?

Concise Answer: A good candidate class has a clear name, cohesive responsibilities, meaningful operations, clear lifecycle, identifiable invariants, and necessary collaborators. It should appear naturally in scenarios and should not exist merely as a container for unrelated data.

30.9 Question 9

Question: What is a mechanism in object-oriented design?

Concise Answer: A mechanism is a collaboration structure among multiple objects that produces higher-level behavior. Classes describe individual object behavior; mechanisms describe

how objects cooperate. Mechanisms are strategic design decisions because they shape system architecture and reuse.

30.10 Question 10

Question: Why are mechanisms more strategic than individual class interfaces?

Concise Answer: An individual class interface is a tactical design decision about one abstraction. A mechanism defines a recurring pattern of collaboration across several abstractions, so it affects coupling, extensibility, testability, and architecture at a larger scale.

30.11 Question 11

Question: Why can structured analysis be risky as a front end to object-oriented design?

Concise Answer: Structured analysis emphasizes functions, transformations, and data flows, while object-oriented design emphasizes responsibilities and collaborating objects. If structured artifacts contain procedural design assumptions, they can bias the object model toward algorithmic decomposition.

30.12 Question 12

Question: What is class promotion?

Concise Answer: Class promotion moves a class or concept upward in a hierarchy when it is more general than originally recognized. It can increase sharing, but if overused it can create overly broad superclasses and weak semantics.

30.13 Question 13

Question: What is a grainsize conflict?

Concise Answer: A grainsize conflict occurs when a class is too general or too coarse for its subclasses, forcing subclasses to inherit irrelevant behavior or bridge a large semantic gap. It is a sign that the hierarchy may need refactoring.

30.14 Question 14

Question: Why is naming important in object-oriented design?

Concise Answer: Names communicate semantics. Good names align code with domain language, reduce cognitive load, and expose unclear responsibilities. Poor names such as **Manager** or **Helper** often hide weak abstractions.

30.15 Question 15

Question: How does MVC illustrate the idea of a mechanism?

Concise Answer: MVC is a mechanism because it defines how model, view, and controller objects collaborate to provide user-interface behavior. The model owns domain state, the view renders it, and the controller handles input. The value is in the collaboration structure, not any single class.

30.16 Question 16

Question: When should you prefer composition over inheritance during classification?

Concise Answer: Prefer composition when variation is behavioral, dynamic, or not based on true substitutability. Inheritance is appropriate when subclasses genuinely satisfy the superclass contract. Composition usually gives more flexibility and avoids fragile hierarchies.

30.17 Question 17

Question: What mistake do candidates often make when explaining object discovery?

Concise Answer: A common mistake is saying “underline nouns and make them classes” as if that is sufficient. Strong candidates explain that noun extraction is only a heuristic and must be validated using responsibilities, scenarios, collaborations, lifecycle, and domain expert vocabulary.

31 Engineering Insights

31.1 Insight 1: Classification Is Design Decision-Making

Classification is not administrative labeling. It is the act of choosing which distinctions matter enough to shape the software.

31.2 Insight 2: Scenarios Are Reality Checks

A class diagram that cannot explain a scenario is probably incomplete or misleading. Scenarios test whether abstractions actually collaborate to deliver value.

31.3 Insight 3: Good Abstractions Have Behavioral Weight

A useful object does not merely store data. It owns responsibilities, preserves invariants, and participates in mechanisms.

31.4 Insight 4: Mechanisms Are Architecture in Motion

Architecture is not only static components. It is also recurring runtime collaboration. Mechanisms capture this dynamic side of architecture.

31.5 Insight 5: Domain Language Is a Design Asset

Using domain terms makes models easier to validate, discuss, and evolve. Vocabulary alignment reduces translation errors between users and developers.

31.6 Insight 6: Classification Should Follow Expected Change

Group things that change together and separate things that change for different reasons. This connects classification directly to maintainability and the Single Responsibility Principle.

31.7 Insight 7: Refactoring Is Part of Classification

When design learning reveals better categories, refactoring is not rework; it is the natural continuation of classification.

32 Connections to Other Topics

32.1 Connection to Abstraction

Classification produces abstractions. Abstraction suppresses irrelevant detail; classification decides which details are irrelevant for a particular purpose.

32.2 Connection to Encapsulation

After classifying responsibilities, encapsulation hides the representation and exposes only the operations required by collaborations.

32.3 Connection to Modularity

Related abstractions form modules or packages. Poor classification often leads to low cohesion and high coupling at the module level.

32.4 Connection to Inheritance

Inheritance is a classification mechanism for expressing generalization and specialization. It should be used when subclass objects are truly substitutable for superclass objects.

32.5 Connection to Polymorphism

Polymorphism depends on classification because different classes must share a meaningful protocol before clients can treat them uniformly.

32.6 Connection to SOLID

- **Single Responsibility Principle:** classification should group responsibilities with one reason to change.
- **Open/Closed Principle:** stable classifications allow new variants without modifying existing code.
- **Liskov Substitution Principle:** inheritance classifications must preserve substitutability.
- **Interface Segregation Principle:** avoid overly broad classifications that force irrelevant operations.
- **Dependency Inversion Principle:** invented abstractions can decouple high-level policy from low-level details.

32.7 Connection to Design Patterns

Design patterns are reusable classifications of collaboration problems and solutions. Patterns such as Observer, Strategy, Command, MVC, and Blackboard encode common mechanisms.

32.8 Connection to Architecture

Architecture depends on classification at larger scales: components, services, bounded contexts, layers, deployment nodes, and frameworks are all classifications of responsibilities.

32.9 Connection to Testing

Classification supports testing by defining equivalence classes, responsibility boundaries, scenario tests, and mockable collaborators.

32.10 Connection to Agile Development

Agile development accepts that classification improves over time. Iterative delivery, refactoring, and domain feedback help models evolve toward better abstractions.

33 Practical Applications

33.1 Application 1: Student Registration System

33.1.1 Candidate Key Abstractions

- Student
- Course
- Section
- Enrollment
- PrerequisitePolicy
- Schedule
- AcademicHold

33.1.2 Important Mechanisms

- Enrollment validation mechanism.
- Seat reservation mechanism.
- Schedule conflict detection mechanism.
- Notification mechanism for waitlist movement.

33.1.3 Design Insight

PrerequisitePolicy is likely an invented design abstraction, while Student, Course, and Section are discovered domain abstractions.

33.2 Application 2: Online Marketplace

33.2.1 Candidate Key Abstractions

- Product
- Seller
- Buyer
- Cart
- Order
- Payment

- Shipment
- ReturnRequest

33.2.2 Important Mechanisms

- Checkout orchestration.
- Payment authorization.
- Inventory reservation.
- Shipment tracking.
- Event logging for audit and analytics.

33.2.3 Design Insight

A naive model may put all checkout logic in `OrderService`. A stronger model distributes responsibilities among `Cart`, `Order`, `PaymentAuthorization`, `InventoryReservation`, and `ShipmentRequest`.

33.3 Application 3: Medical Monitoring System

33.3.1 Candidate Key Abstractions

- Patient
- Sensor
- Reading
- Threshold
- Alert
- Clinician
- CareUnit

33.3.2 Important Mechanisms

- Sensor ingestion.
- Threshold evaluation.
- Alert escalation.
- Event logging.
- Dashboard update via dependency notification.

33.3.3 Design Insight

This system requires careful classification of events, states, and responsibilities because reliability, safety, latency, and auditability are architecture-driving concerns.

33.4 Application 4: Drawing Application

33.4.1 Candidate Key Abstractions

- Shape

- Canvas
- Selection
- Tool
- Layer
- Command

33.4.2 Important Mechanisms

- Rendering mechanism.
- Selection mechanism.
- Undo/redo mechanism.
- Tool dispatch mechanism.

33.4.3 Design Insight

Command is an invented abstraction that supports undo/redo. It may not appear in user language, but it is essential to the design mechanism.

34 Summary Cheat Sheet

Concept	High-Value Summary
Classification	Grouping things by meaningful sameness for a design purpose.
Discovery	Finding domain abstractions already present in user vocabulary.
Invention	Creating solution abstractions needed for implementation or architecture.
Classical categorization	Classification by necessary and sufficient properties.
Conceptual clustering	Classification by fit to a concept, behavior, or responsibility.
Prototype theory	Classification by resemblance to a representative example.
Object-oriented analysis	Discover problem-domain classes and objects.
Object-oriented design	Invent solution abstractions and mechanisms.
Use case	Actor-centered sequence of behavior that produces measurable value.
Scenario	Concrete path through a use case; excellent for finding responsibilities.
CRC card	Lightweight card showing class, responsibilities, and collaborators.
Domain analysis	Finding common abstractions and mechanisms across a domain.
Key abstraction	Important class or object in the problem or solution vocabulary.
Mechanism	Collaboration structure among objects that provides higher-level behavior.
Idiom	Low-level language or culture-specific convention.
Framework	Reusable collection of classes and mechanisms for a domain.
Class promotion	Moving a concept upward in a hierarchy to increase sharing.
Grainsize conflict	Class is too general/coarse, making subclasses awkward.

MVC	Mechanism separating model state, view rendering, and controller input.
Blackboard	Mechanism where agents collaborate through a shared problem-solving workspace.
Best test of a model	It explains scenarios, assigns responsibilities cleanly, and localizes change.

Final Exam Memory Hook

Classification finds the classes. Scenarios validate the responsibilities. Mechanisms explain the collaborations. Refactoring improves the model as understanding grows.